CMSC 475: Applications of Audio Classification on Music Recomendation

Aren Vista, Haliey Davio
University of Maryland, Baltimore County
May 14, 2026

1 Introduction

In order to maximize user retention, modern day music streaming services prioritize effective but lightweight algorithms to determine which songs get recommended to which users. Some rely on cursory music metadata: if a user likes a song from a particular artist, for example, the algorithm might suggest another song from that same artist. Many platforms also base their suggestions on the listening habits of other similar users: if two users have similar song histories, the platform may recommend their songs to each other. These approaches do not always guarantee that the suggested song matches the rest of a user's preference in areas like genre or tone. Moreover, they may rely heavily on other users' histories and patterns. In contrast, we aimed to design a more independent recommendation process that could perform well even without using other users' behaviors to contextualize an individual user's preferences.

2 Design Goal

In general, we found that large platforms often employ more simplistic audio analysis in order to keep their applications lightweight and streamline the user experience. This simplicity, however, often comes at the cost of feature-based accuracy. In our project, we aimed to take a more in-depth look at what defines music and how that definition relates to listening preferences. We examined other data channels and interpretations in order to create a comprehensive, multi-dimensional approach to evaluating music tracks.

By shifting our focus from who made the song or how it is categorized to what it actually sounds like, we bridge the gap between deep acoustic characteristics and user preferences. We believe this approach not only yields highly accurate recommendations but also naturally surfaces independent or lesser-known artists who might otherwise be buried by traditional, popularity-driven algorithms.

To achieve this, we focused on three objectives:

1. *Classifying Audio via Spectrogram Analysis*: The system will convert raw audio waveforms into spectrograms. This allows the audio data to be treated as analogous to image data, e.g. visual representations of a track's frequencies and amplitudes over time. By treating audio as an image, the system can utilize deep learning models (such as Convolutional Neural Networks) to extract complex acoustic features like timbre, tempo, and harmonic structure.
2. *Building User Profiles*: Rather than relying on a user's clicked genres or liked artists, the system will compile and analyze the spectrogram data directly from the user's listening history. This aggregate data will be used to construct a unique, personalized "profile" that accurately maps the user's auditory preferences.
3. *Generating Content-Based Recommendations*: Using this profile, the system will compare the user's acoustic baseline against a database of unlistened tracks. By mathematically matching the acoustic signatures, the engine will suggest new songs that authentically resonate with the user's established tastes based purely on similarity.

3 Data Collection

Labeled audio tracks will be sourced from FMA: A Dataset For Music Analysis [2]. The audio data is MP3-encoded. Each song will be converted from .mp3 format to a spectrogram to be processed as visual data. Other audio data provided by the dataset (volume, song length, etc.) may also play a role in data processing.

3.1 Metadata

Similar to existing music streaming platforms, the model takes into account each track's metadata. The genre is a simple way to get the general impression

of a track. Trends and grouping may be gleaned from information about the artist and album. Tags provide supplementary information. The number of listens provides insight into whether popular songs have more in common with each other than their unpopular counterparts. These data points, alongside many others contained in the dataset, provide a strong baseline for song analysis.

3.2 Lyrics

While metadata can provide general details about a song, lyrics provide more insight into semantic meaning and tone. This was found to be especially useful because these aspects are rarely captured by other song recommendation algorithms. Though this approach did come with its own limitations, it generally gave the model a fuller picture of each track.

4 Methods

Our method consisted of four main stages: dataset preparation, audio representation with the Audio Spectrogram Transformer (AST), multimodal alignment with CLIP, and dimensionality reduction with principal component analysis (PCA).

4.1 Dataset

Primarily used the Free Music Archive (FMA) dataset [2]. FMA contains 106,574 tracks from 16,341 artists and 14,854 albums, spanning 161 genres. Training on such a diverse collection encouraged the model to generalize more effectively and produce embeddings for previously unheard audio.

For each track, FMA provides a high-quality MP3 file, more than 500 precomputed features, and extensive metadata, including title, album, artist, duration, genre, and release date [2]. For our experiments, we selected the large subset of the dataset, which yielded over 100,000 songs. Each track was trimmed to 30 seconds as a preprocessing step, allowing us to maintain consistent model inputs when converting MP3 files into a visual representation.

4.2 Audio Representation with AST

Our training pipeline began with a pretrained Audio Spectrogram Transformer (AST) developed by Gong et al. [**?**]. The AST first converts each MP3 file into a spectrogram, a visual representation of audio that captures changes in time, frequency, and amplitude. Compared with alternative visual formats such as waveforms, spectrograms preserve greater detail and are therefore better suited for deeper audio analysis [**?**].

Because every input clip was limited to 30 seconds, each spectrogram shared the same overall scale and axes, producing more uniform training data. Although these shorter clips were easier to process, they did not fully exploit the AST's ability to handle variable-length inputs. This strict consistency may also have reduced the model's ability to generalize, which may have contributed to weaker performance on testing data that did not follow the same 30-second structure.

The AST divides each spectrogram into overlapping 16×16 image patches and linearly projects them into one-dimensional patch embeddings. These embeddings are combined with learnable positional embeddings and prepended with a classification token. The resulting sequence is then passed through the Transformer, while the classification token is used with a linear layer for classification tasks [**?**]. This architecture provides an effective mechanism for processing audio through a visual input format.

The pretrained AST weights were originally fine-tuned on AudioSet, a large labeled dataset developed by Google that contains over two million audio clips [**?**]. Because AudioSet primarily consists of sound events collected from YouTube rather than music-specific examples, the pretrained model required additional fine-tuning to better suit our music-domain task.

4.3 Multimodal Alignment with CLIP

The second major component of the multimodal framework was Contrastive Language–Image Pretraining (CLIP) [4]. While AST handled the audio-based visual representation, CLIP enabled the incorporation of textual and semantic information. For each song containing vocals, a corresponding set of lyrics was generated and passed into CLIP to produce a 512-dimensional lyric embedding. The 768-dimensional audio embedding produced by AST was then compressed to 512 dimensions so that it matched the lyric embedding size.

A symmetric cross-entropy loss was then computed to align each audio embedding more closely with its corresponding lyric embedding. Following Radford et al., a temperature parameter was used to give the model flexibility in how it measured distances and formed clusters [4]. After computing the loss, backpropagation propagated the error through the network to determine gradients and reduce the loss during subsequent iterations. As is standard for Transformer-

based models, AdamW was used to update the model weights through a combination of momentum, adaptive scaling, and weight decay.

4.4 Principal Component Analysis

The resulting 512-dimensional embeddings were centered and reduced in preparation for principal component analysis (PCA). The principal axes—PC1, PC2, and PC3—were computed to capture the greatest variance among the data points, and all output vectors were then projected into this PCA space.

As a linear dimensionality reduction method, PCA provided a more interpretable representation of the data while reducing the risk of spurious or misleading clusters that can arise with nonlinear methods such as UMAP or t-SNE.

5 Related Work

5.1 MIT AST

The Audio Spectrogram Transformer (AST), implemented for interpreting audio data as spectrogram images, is a purely attention-based model used for audio classification. It was conceptualized as a way to test whether reliance on a CNN was necessary in the domain of audio classification and how domain tasks could be accomplished with a convolution-free model. Historically, CNN-attention hybrids have been used to meet similar goals. It was common to see CNNs with self-attention mechanisms layered on top excelling at audio classification and speech and emotion recognition. The AST mirrors the Vision Transformer (ViT) for vision tasks, using a simpler Transformer architecture with fewer parameters. Notably, while the AST's input lengths ranged from 1 to 10 seconds, the inputs used in this work were consistently 30 seconds. While audio tasks often suffer from a shortage of available data compared to image tasks, AST was able to address that limitation by using a modified, pretrained Vision Transformer (ViT). This modified version condensed the 3-channel input of ViT into a single channel for AST and adapted positional embeddings for variable-length input, resulting in a much quicker training process when compared to randomized initialization.

5.2 Spotify

When outlining the goals of the project, existing music platforms were used as a baseline for comparing song recommendation methods and results. While Spotify, one of the primary inspirations for this work, has no public documentation on the algorithm it uses for music recommendation, its approach has been speculated to include multiple machine learning algorithms. One of its major algorithm components is content filtering, which involves recording and analyzing the habits of existing listeners. Using these learned habits, the platform can more reliably predict the behaviors and listening habits of similar users. This filtering uses two methods: one is an "exploitative" method that suggests songs based on existing data, preferences, and habits of similar users; another is an "explorative" method based more heavily on track features, which suggests songs that contrast existing preferences. Natural Language Processing is likely another facet of Spotify's algorithm, wherein text from song titles, album titles, tags, and searches are used to predict user preferences. It is believed that audio models are also used to examine the audial properties of songs, incorporating that information into recommendations. A combination of these approaches blends to create an effective song recommendation system with high user retention, highlighting both potential parallels and contrasts for this project. [1]

5.3 FMA

The Free Music Archive (FMA) dataset used for training the model was derived from an open library of the same name. Curated by WFMU, a long-running U.S. radio station, this dataset highlights legally accessible song tracks and has established itself as a large-scale, high-quality library for audio-based projects. It sets itself apart from many other music datasets, such as OMRAS2 and Codaich, by containing the actual audio files. Previously, the standard dataset for Music Information Retrieval (MIR) was GTZAN, which contained only 1,000 clips from 10 genres. In contrast, FMA contains over 100,000 clips from over 160 genres. It also contains a wealth of metadata, with metrics from the actual songs in addition to user interactions. It works especially well for Music Genre Retrieval (MGR) due to its feature organization. Features include several sub-genres as well as a built-in genre hierarchy that follows original track annotations. A collection of over 500 pre-computed features also provides valuable insight into track identification. A combination of these attributes makes the dataset exceptionally useful for a variety of tasks within the music domain. [2]

5.4 CLIP

Contrastive Language-Image Pretraining (CLIP) is a model designed to align image and text embeddings in a purely attention-based format. Its effectiveness can, in part, be attributed to its massive dataset. The development team cultivated a new dataset of 400 million pairs of images and text, forming a diverse collection of information and greatly reducing the risk of over-fitting. Pretraining approaches with both ResNet-50 and a slightly modified version of the Vision Transformer (ViT) were contemplated. Both versions were run for 32 epochs to measure and compare performance between the two image encoders. The text encoder is a 63M-parameter, 512-wide Transformer with architectural modifications. A decayed learning rate, Adam optimizer, and decoupled weight decay regularization were all used during training. Initial hyperparameters were based on a combination of grid and random searches with manual tuning. The model was ultimately trained to predict text and image pairings, enabling flexible zero-shot transfer that outperformed many comparable approaches like ViR-M and SimCLRv2. The team found success in engineering the text associated with each image to be more specific and context-focused. The model also performed well when it came to representation learning. Though the performance of smaller CLIP models was more reliant on their training dataset, the models scale well and are able to learn a broader spectrum of tasks. A combination of custom architecture and training ultimately made CLIP a robust model that excelled even when compared to human performance.

5.5 CLAP

Another model prevalent in the MIR space is the audio alternative to CLIP, CLAP (Contrastive Language-Audio Pretraining). CLAP was only discovered toward the end of the project's development and therefore could not be implemented, but it aligns well with the project's goals by introducing a clean method of directly aligning audio with text. It aims to emulate the way that humans are able to recognize and contextualize audial input in the same way that CLIP does with images. It has been trained to work with multiple domains, from Sound Event Classification to speech-related tasks. The model also had applications in this domain, addressing music tasks like genres, strokes, and tonics. It employs zero-shot predictions, avoiding the limitations of training with class labels and strict predictions at inference time. CLAP utilizes two encoders and contrastive learning in order to bring text and audio data into a multimodal space, essentially combining the manual chaining of the ViT and CLIP embeddings used in this work. The two encoders process audio and text, learning the embedding space based on the inputs' similarities and differences. Once learned, CLAP is capable of flexible class predictions and a variety of downstream tasks.

5.6 Limitations and Continuation

Language often contradicts a song's overall tone. Incorporating the semantic content of lyrics introduced an additional challenge. Because music is inherently multi-dimensional and open to interpretation, the meaning conveyed by the lyrics does not always align with the song's other characteristics. For example, an upbeat pop track may feature melancholic or introspective lyrics, producing a semantic representation that appears inconsistent with the song's genre, mood, or instrumentation.

Such mismatches can weaken the effectiveness of the contrastive learning approach [4, 3]. In particular, when lyrical semantics conflict with the broader musical context, the resulting embeddings may capture contradictory signals, making it more difficult for the model to learn coherent cross-modal relationships.

User data was difficult to obtain. There was no straightforward way to query popular music platforms such as Spotify [1] for their top playlists. To evaluate the song recommendation algorithm more realistically, the sample user profiles were therefore compiled by hand.

For each "user," a set of songs within the same genre, or at least similar enough to approximate a coherent listening preference, was selected. The model was trained on 30-second audio clips from the FMA dataset [2], so each selected song also had to be trimmed to a 30-second excerpt to ensure the best possible compatibility with the training data.

Given more time, a useful extension would be to automate this preprocessing step by writing a script that segments longer tracks into 30-second clips and averages the resulting predictions. However, due to time constraints, this process was performed manually.

Large dataset download issues. Downloading a large dataset proved unreliable, as the `wget -c` command repeatedly stalled at 0-byte transfers. After ruling out problems with the command itself, it was determined that the issue was likely related to the selected partition. Subsequent correspondence with HPC staff helped diagnose the problem and identify a workable solution.

Missing FFmpeg dependency for multicore jobs. Training required `libcodec.dll` from `FFmpeg`. Although this dependency was available on the single-core cluster, the necessary module was not available for multicore jobs, as confirmed by checking the output of `module avail`. To work around this limitation, a static build was downloaded from the Git repository and the executable was added to the `PATH` environment variable.

Acquiring a lyrics dataset was challenging. A centralized repository containing lyrics for all tracks in the collection could not be found. In addition, some tracks, such as instrumentals, do not contain lyrics at all, which introduced an additional design consideration when constructing the dataset.

Given more time, a promising extension would be to develop a representation that captures the semantic content conveyed by musical notes themselves. For example, one could draw on ideas from music theory to group chords, motifs, or harmonic progressions and model them as a "corpus" analogous to natural language, thereby enabling analysis even for tracks without lyrics.

References

[1] XDA Developers. *Spotify Algorithm.* https://www.xda-developers.com/spotify-algorithm-recommends-music/.

[2] Michaël Defferrard, Kirell Benzi, Pierre Vandergheynst, and Xavier Bresson. *FMA: A Dataset for Music Analysis.* In *International Society for Music Information Retrieval Conference (ISMIR)*, 2017. https://github.com/mdeff/fma.

[3] Yusong Wu, Ke Chen, Tianyu Zhang, Yuchen Hui, Taylor Berg-Kirkpatrick, and Shlomo Dubnov. *Large-scale Contrastive Language-Audio Pretraining with Feature Fusion and Keyword-to-Caption Augmentation.* In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2023.

[4] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. *Learning Transferable Visual Models From Natural Language Supervision.* In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, pages 8748–8763, 2021.h Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. *Learning Transferable Visual Models From Natural Language Supervision.* In *Proceedings of the 38th International Conference on Machine Learning (ICML)*,